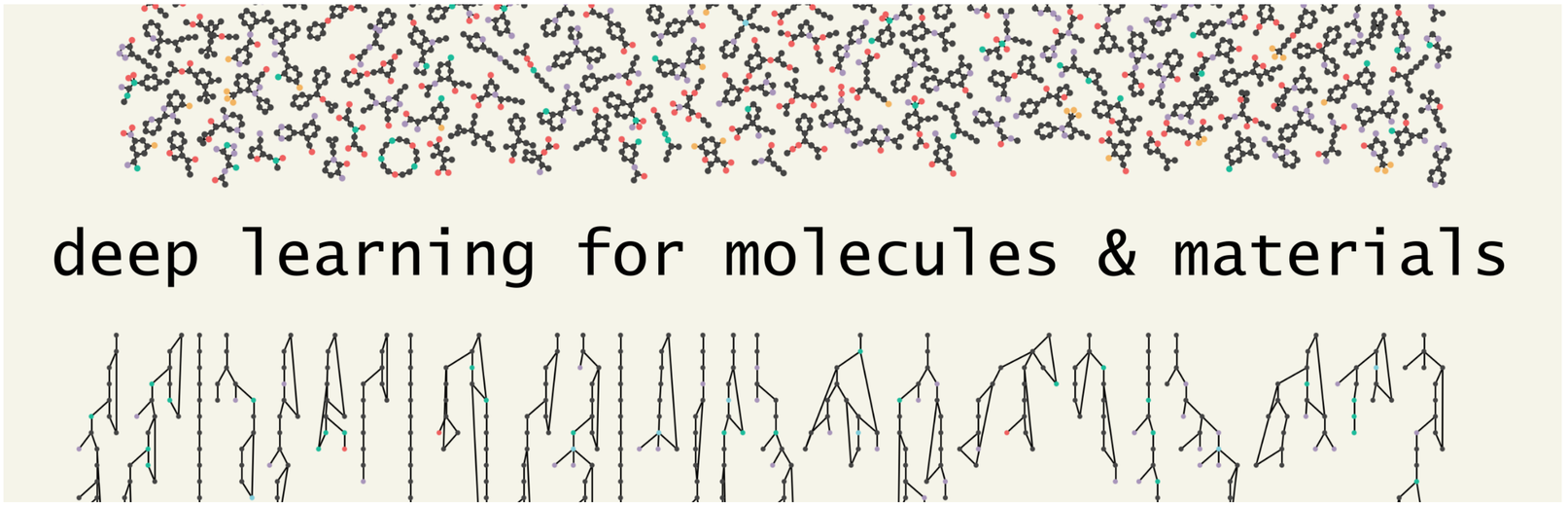
Introduction to neural network



Surajit Das
I.PhD. Chemistry
26 Oct 2026



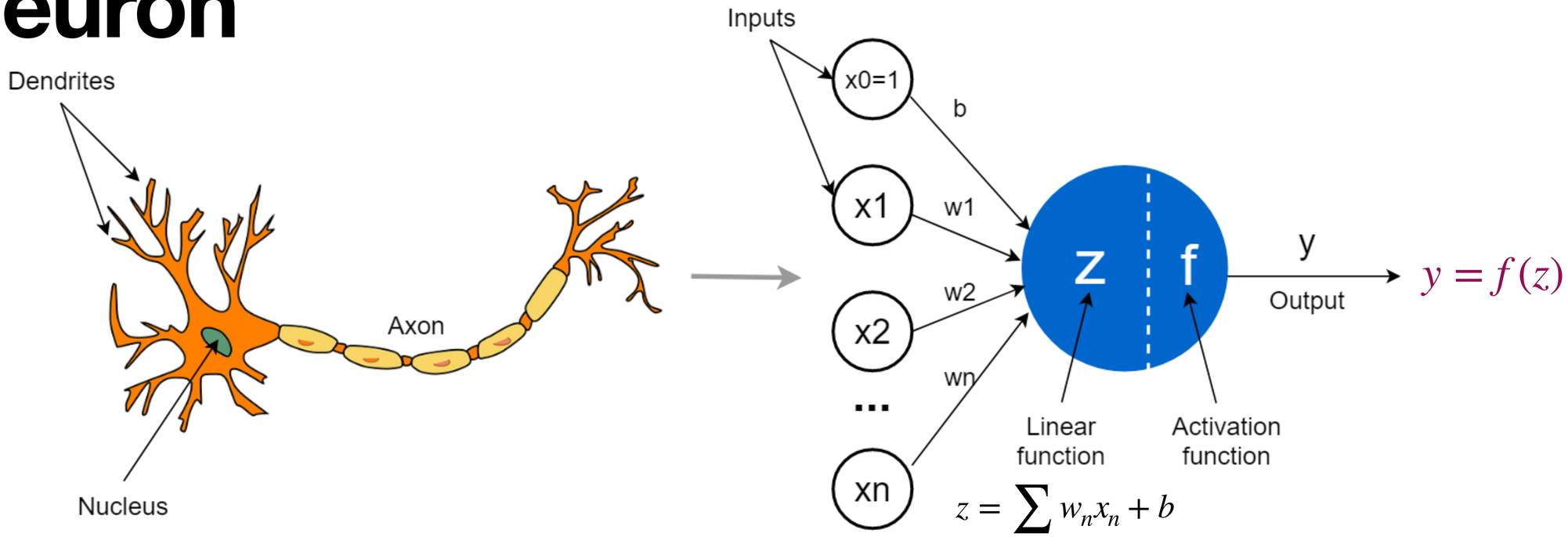

```
@article{white2021deep,
 title={Deep Learning for Molecules and Materials},
 journal={Living Journal of Computational Molecular Science},
 author={White, Andrew D},
 url={https://dmol.pub},
 year={2021},
 volume={3},
 number={1},
 pages={1499},
 doi={10.33011/livecoms.3.1.1499}
}
```

https://dmol.pub/index.html

C. Deep Learning

6. Deep Learning Overview
7. Standard Layers
8. Graph Neural Networks
9. Input Data & Equivariances
10. Equivariant Neural Networks
11. Explaining Predictions
12. Attention Layers
13. Deep Learning on Sequences
14. Variational Autoencoder
15. Normalizing Flows
16. Modern Molecular NNs

Neuron



$$y = f\left(\sum w_n x_n + b\right)$$

x_n are the input features

w_n are the **weights** Parameter

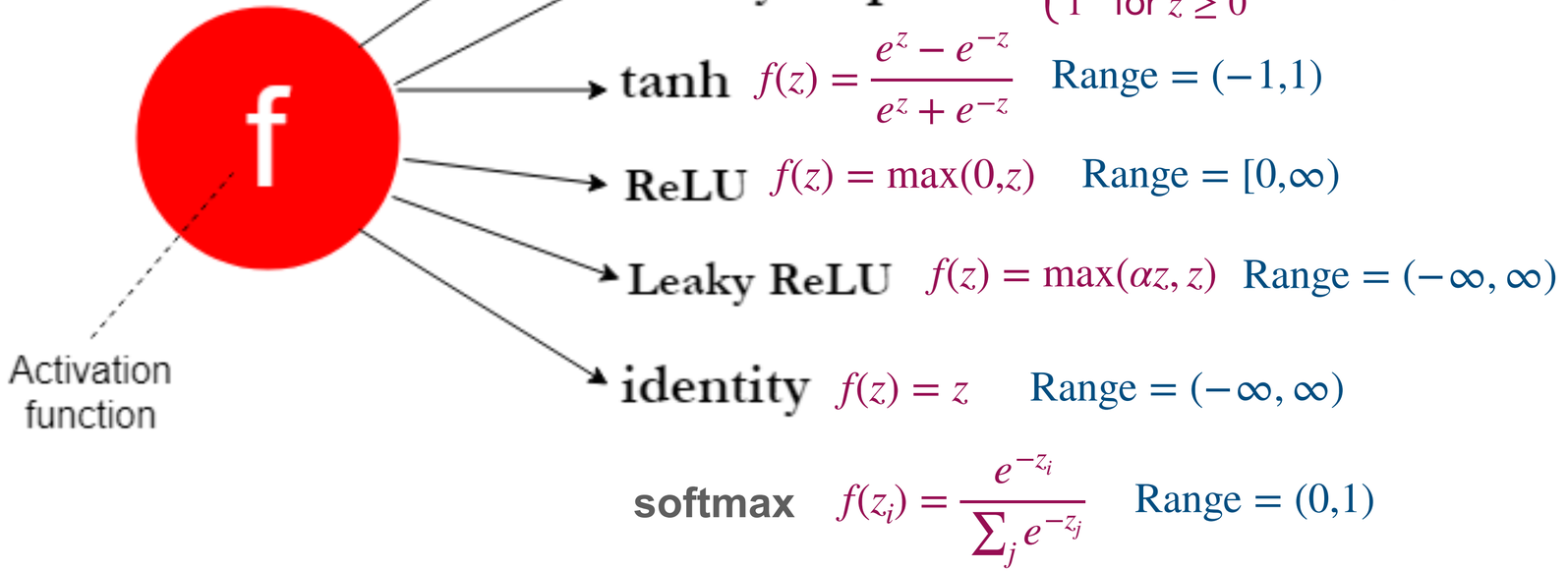
b is the **bias** Parameter

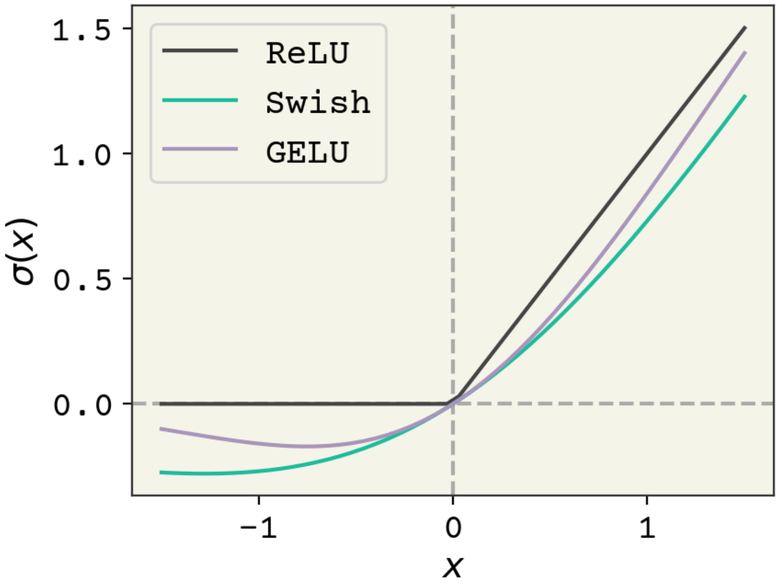
$f(\cdot)$ or $\sigma(\cdot)$ is the **activation function** Hyper-parameter

https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

Neuron

Activation functions

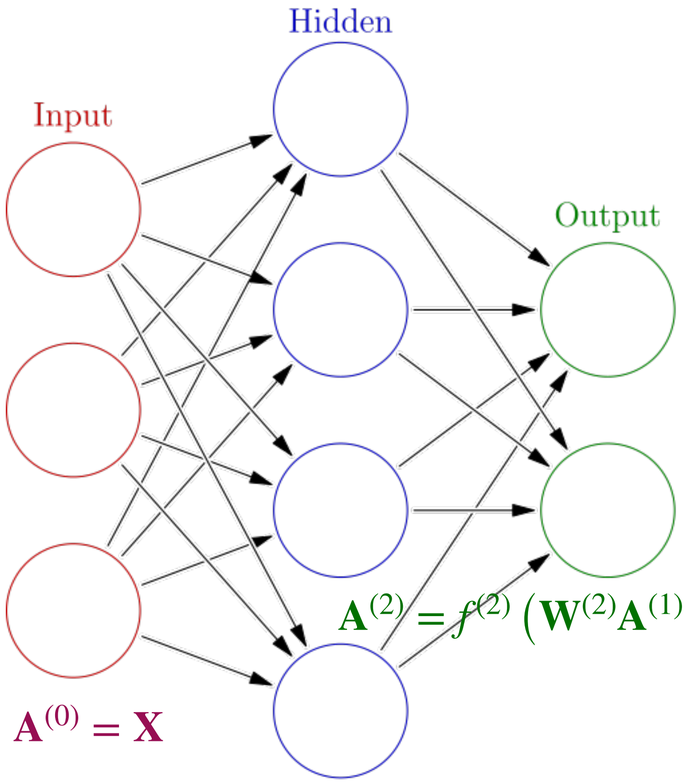




https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

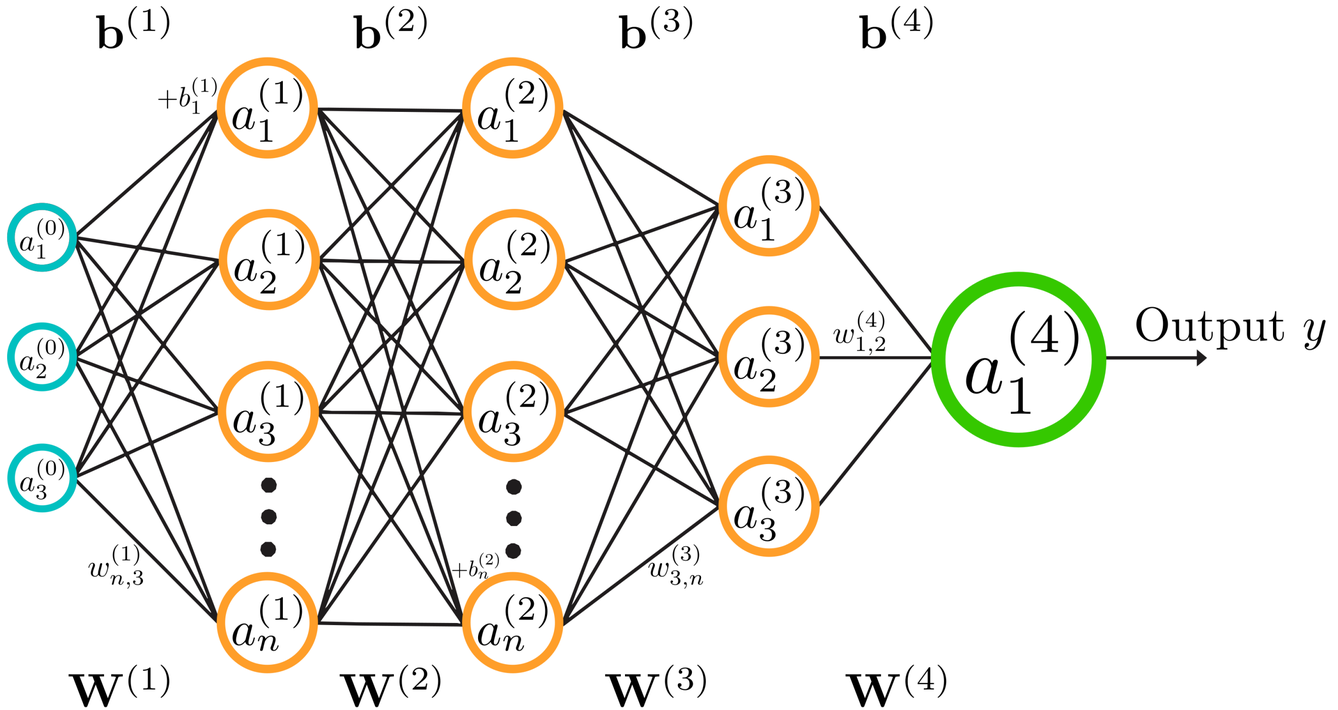
Neural Layers

Dense** (fully-connected) **neural network



Input $\mathbf{x}$

Feedforward neural network



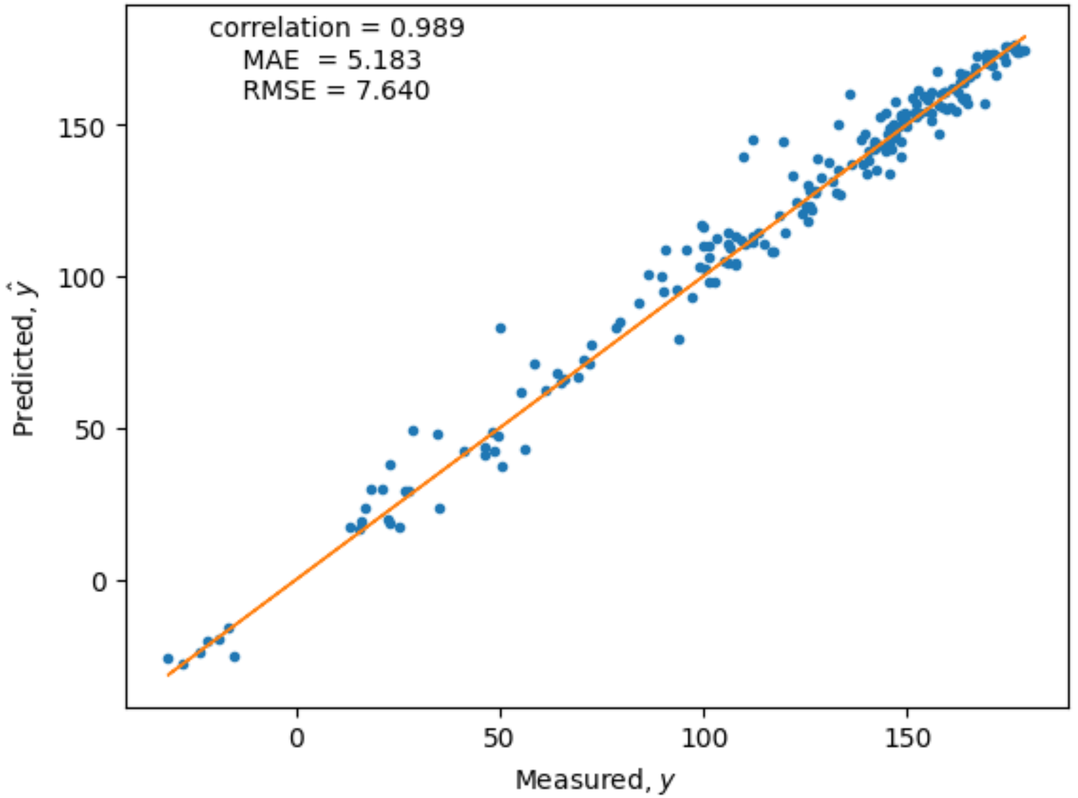
$$\mathbf{A}^{[1]}_{F\times m} = f\left(\mathbf{W}^{[1]}_{F\times D}\mathbf{A}^{[0]}_{D\times m} + \mathbf{b}^{[1]}_{F\times m}\right)$$

D is the input dimension
F is the output dimension

https://en.wikipedia.org/wiki/Neural_network_%28machine_learning%29

Prediction of $\sigma_{\mathrm{iso}}(^{13}\mathrm{C})$ using $\mathrm{aBoB-RBF(0)}$ as input layer

```
model = nn.Sequential(
    nn.Linear(X.shape[1], 1024),
    nn.ReLU(),
    nn.Linear(1024, 512),
    nn.Tanh(),
    nn.Linear(512, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.Tanh(),
    nn.Linear(64, 1),
).to(device)
```



Loss functions: $\mathscr{L}(y, \hat{y})$
nn.L1Loss()
nn.MSELoss()
nn.CrossEntropyLoss()

Optimizers:
torch.optim.SGD
torch.optim.Adam
torch.optim.RMSprop

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \nabla_{\mathbf{w}} \mathscr{L}$$

Hyperparameter: parameters that can be tuned, but they are **not** trained on data.

- Continuous ⇒ regularisation strength
- Categorical ⇒ activation function
- Discrete ⇒ Number of layers

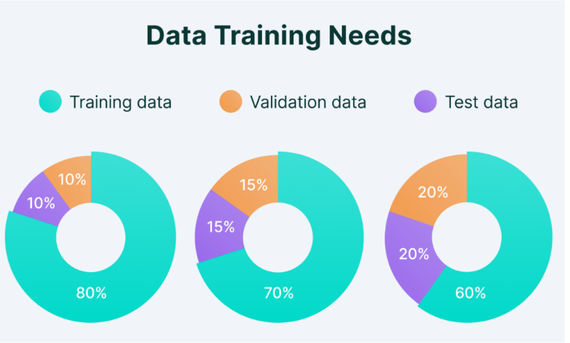
Universal Approximation Theorem:

"*… neural networks with a certain structure can, in principle, approximate any continuous function to any desired degree of accuracy.*"

Stinchcombe & White, *Neural networks*, *2*(5), 359-366, 1989.

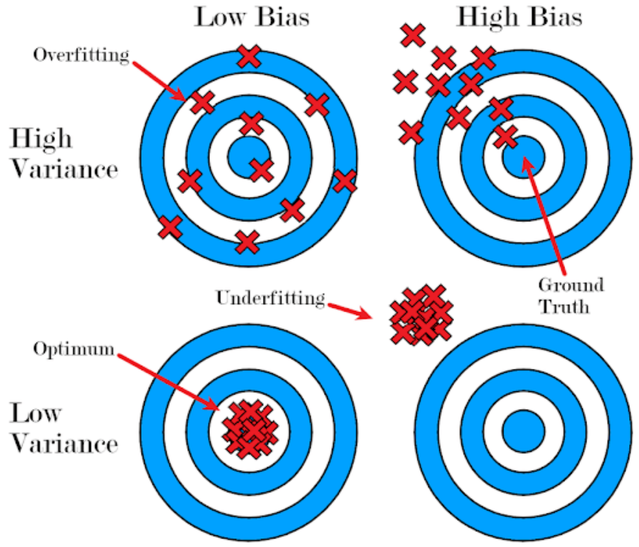
Hyperparameter tuning:

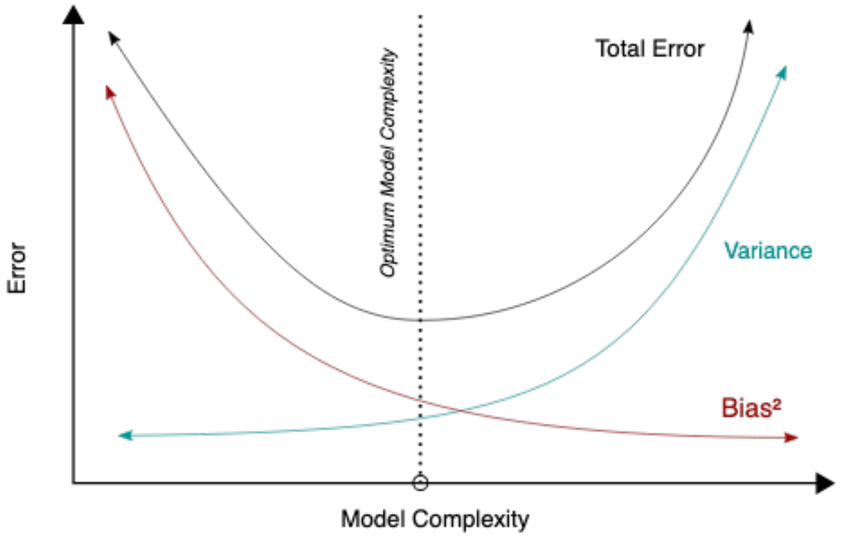
- Meta-learning
- auto-ML
- τ-plot (for regression models)

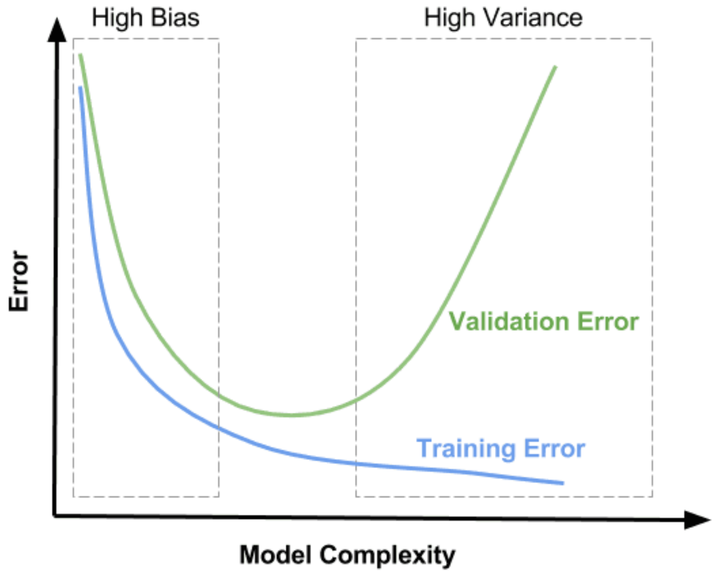


https://www.analyticsvidhya.com/

Bias-variance tradeoff:

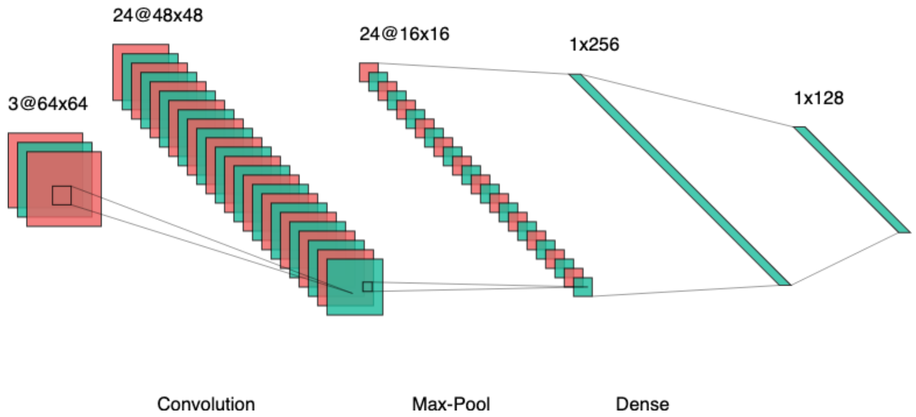






wikipedia, google

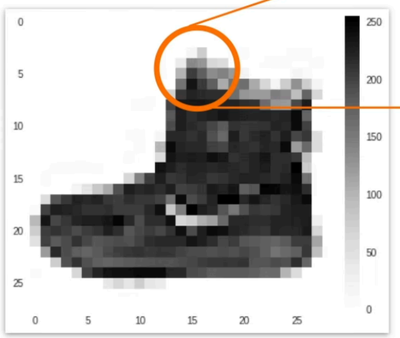
#Chapter 7 Standard Layers



Dense or fully-connected layer ✓

Convolution layer

- Learn something independent of the location on the input grid.
- Kernel, a set of trainable multipliers, whose aim in to learn the features from the input data.
- Number of kernels is termed filters.



0	64	128
48	192	144
142	226	168

Current pixel value is 192

Consider neighbor values

-1	0	-2
.5	4.5	-1.5
1.5	2	-3

Filter definition

```
CURRENT_PIXEL_VALUE = 192
NEW_PIXEL_VALUE = (-1 * 0) + (0 * 64) + (-2 * 128) +
                  (.5 * 48) + (4.5 * 192) + (-1.5 * 144) +
                  (1.5 * 42) + (2 * 226) + (-3 * 168)
```

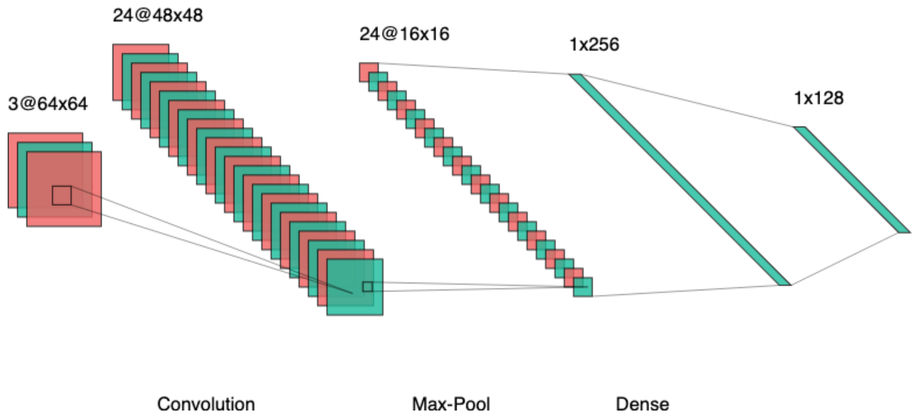
$$(B, H, W, C) \rightarrow (B, \approx H, \approx W, F)$$

For example, $(16,128,128,3)$ when applied with single filter of $5 \times 5 \times 3$ kernel the new dimension will will be $(16,124,124,1)$

For real use, multiple filters are used with multiple convolution layers to capture various features of the data. For example, vertical edges, horizontal gradients, texture, colour etc.

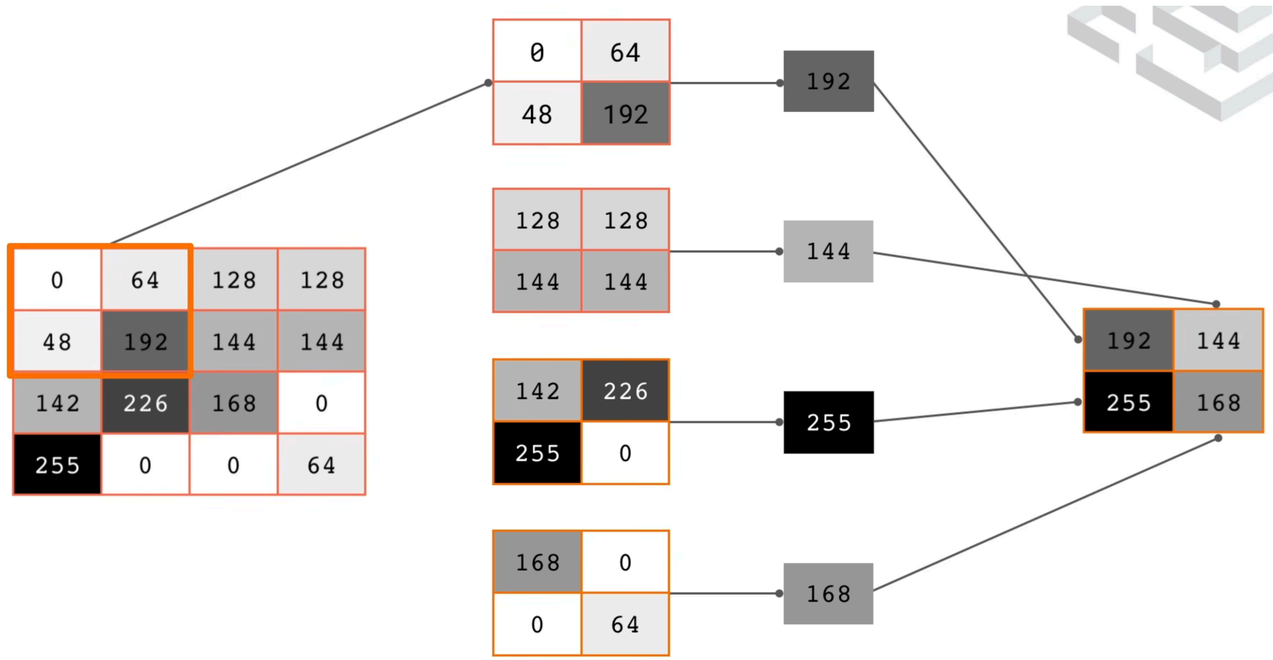
https://youtu.be/x_VrgWTKkiM?si=0N3ZuIlPorhsGb36
https://youtu.be/YGILT182T6w?si=vvlIEwI9YJG9hEnS

#Chapter 7 Standard Layers



Pooling

- Pooling layers are applied after convolution layers
- Apply windows, to reduce the dimension
- Average and maximum
- $(B, H, W, F) \rightarrow (B, H/k, W/k, F)$



Invariance **Equivariance**

*"Convolutions and pooling should be chosen when you want to have translation **invariance**."*

*"Convolutions are actually translationally **equivariant** …"*

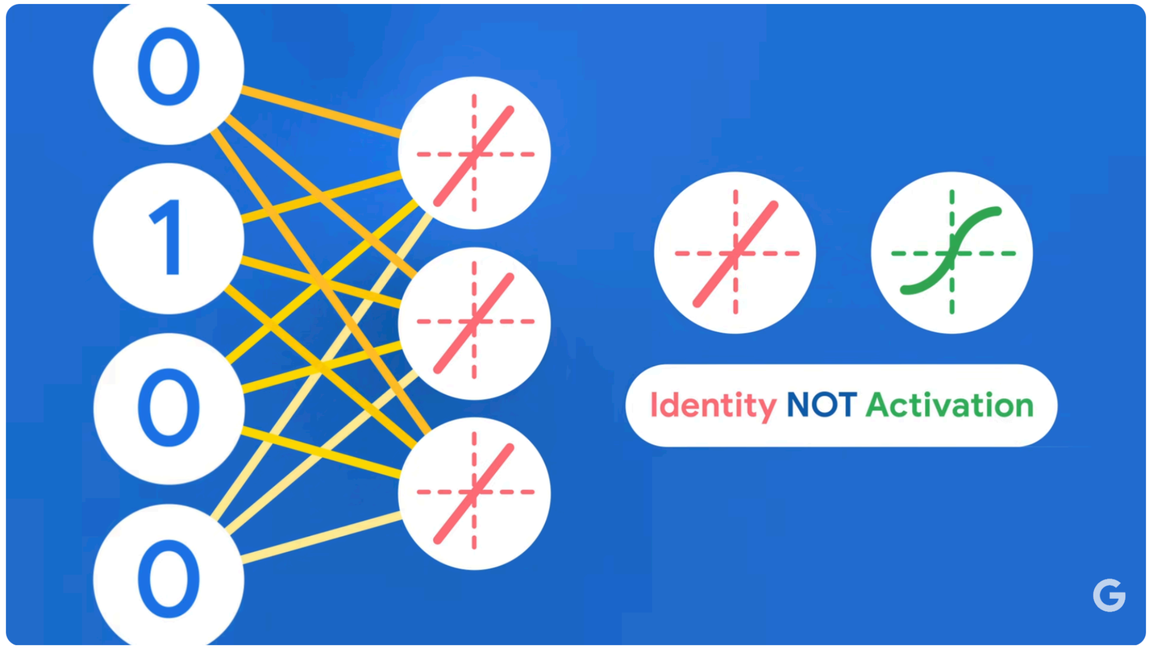
https://youtu.be/x_VrgWTKkiM?si=0N3ZuIlPorhsGb36

#Chapter 7 Standard Layers

Embedding

- Characters to **numerical vectors**
- Large discrete space to small space of real numbers

⇒ Character **hello**
⇒ Tokens [‘**h**’, ‘**e**’, ‘**l**’, ‘**l**’, ‘**o**’]
⇒ Indices via. dictionary [7,4,11,11,14]
⇒ Embedding layer



https://www.youtube.com/watch?v=my5wFNQpFO0

If a protein is soluble ?

```
# 20 amino acids and 0 for padding.
# Each protein have 200 amino acid sequence (with padding)
```

```
SolubilityNetDropout(
  (embedding): Embedding(21, 2, padding_idx=0) # ⇒ ——————————————————————————————(B, 200, 2)
  (conv1): Conv1d(2, 16, kernel_size=(5,), stride=(1,)) # ⇒ ———————————————————————(B, 196, 16)
  (pool1): MaxPool1d(kernel_size=4, stride=4, padding=0, dilation=1, ceil_mode=False) # ⇒ —(B, 49, 16)
  (conv2): Conv1d(16, 16, kernel_size=(3,), stride=(1,)) # ⇒ ———————————————————————(B, 47, 16)
  (pool2): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False) # ⇒——(B, 23, 16)
  (conv3): Conv1d(16, 16, kernel_size=(3,), stride=(1,)) # ⇒ ———————————————————————(B, 21, 16)
  (pool3): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False) # ⇒——(B, 10, 16)
  (flatten): Flatten(start_dim=1, end_dim=-1) # ⇒ ———————————————————————————————(B, 160)
  (fc1): Linear(in_features=160, out_features=256, bias=True) # ⇒ ——————————————————(B, 256)
  (fc2): Linear(in_features=256, out_features=64, bias=True) # ⇒ ———————————————————(B, 64)
  (fc3): Linear(in_features=64, out_features=1, bias=True) # ⇒ ————————————————————(B, 1)
  (relu): ReLU() # ⇒ ——————————————————————————————————————————————————————(B, 1)
  (sigmoid): Sigmoid() # ⇒ ————————————————————————————————————————————————————(B, 1)
  (d1): Dropout(p=0.3, inplace=False)
  (d2): Dropout(p=0.3, inplace=False)
  (d3): Dropout(p=0.3, inplace=False)
)

Total trainable parameters: 59,515
```